

SUPMEAL

DOCUMENTATION TECHNIQUE

Application de gestion de recettes, cookbooks et plans de repas

Projet	SUPMEAL
Année	2026
Type	Sujet de rattrapage
École	SUPINFO Tours
Auteur	MOUTOUSSAMY Oriane

Sommaire

Sommaire	2
1. Contexte du projet	3
2. Objectifs fonctionnels	4
3. Stack technique	4
4. Architecture globale	5
5. Backend	6
5.1. Structure du backend	6
5.2. Dépendances et rôle des bibliothèques	6
5.3. Gestion des erreurs	6
5.4. Validation des entrées	7
5.5. Authentification et autorisations	7
5.6. API REST	7
5.7. Swagger	8
6. Frontend	9
6.1. Setup et environnement	9
6.2. Pourquoi React + Vite ?	9
7. Base de données et schéma de données	10
7.1. Relations principales	10
8. Guide de démarrage	11
8.1. Avec Docker Compose	11
8.2. En développement local	11
9. Sécurité et configuration	12
10. Points forts et limites du projet	13
10.1. Points forts	13
10.2. Limites	13
11. Note sur le rendu incomplet	14

1. Contexte du projet

Le projet a été réalisé dans le cadre d'un sujet de rattrapage. Il a été conçu comme un MVP fonctionnel démontrant les bases d'une architecture backend/frontend moderne, structurée et cohérente.

- Créer et gérer des recettes.
- Organiser des cookbooks.
- Inviter d'autres utilisateurs.
- Gérer des plans de repas.
- Exporter et importer ses données.
- Utiliser une API documentée et testable.

2. Objectifs fonctionnels

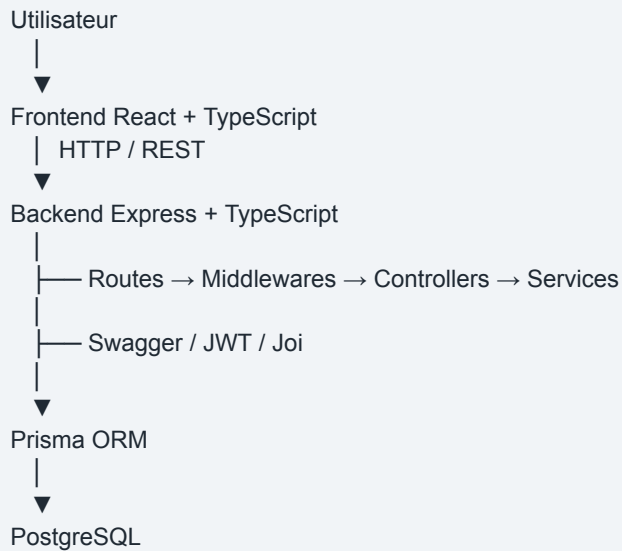
- Centraliser les recettes et les informations nutritionnelles de manière structurée.
- Gérer des cookbooks avec accès par utilisateur.
- Définir des rôles au sein d'un cookbook : [OWNER](#), [EDITOR](#), [COMMENTER](#), [READER](#).
- Proposer un flux de création et de modification de recettes.
- Gérer les favoris et les commentaires.
- Organiser un planning de repas.
- Exposer une API REST avec documentation Swagger.
- Sécuriser l'application avec JWT et validation des entrées.
- Permettre le démarrage avec Docker afin de simplifier le déploiement local.

3. Stack technique

Couche	Technologies	Rôle principal
Backend	Node.js, TypeScript, Express, Prisma ORM, JWT, Joi, Swagger, Socket.IO	API, logique métier, sécurité et accès aux données
Base de données	PostgreSQL	Persistance relationnelle
Frontend	React, TypeScript, Vite, Nginx	Interface utilisateur et build de production
Environnement	Docker Compose, Git, npm, .env	Exécution, versionnement et configuration

4. Architecture globale

L'architecture sépare clairement les responsabilités entre l'interface, l'API et la persistance des données.



- Le frontend React consomme l'API backend.
- Le backend expose des endpoints REST sécurisés.
- Prisma gère la couche de persistance vers PostgreSQL.
- Swagger fournit une documentation interactive des routes.
- Docker Compose orchestre la base de données, le backend et le frontend.

Le backend est organisé autour de contrôleurs, middlewares, routes, services, validateurs, configuration et accès à la base de données.

5. Backend

5.1. Structure du backend

Emplacement	Responsabilité
<code>src/app.ts</code>	Initialisation de l'application Express
<code>src/server.ts</code>	Démarrage du serveur HTTP
<code>src/routes/index.ts</code>	Définition de toutes les routes
<code>src/controllers/</code>	Logique métier exposée par les endpoints
<code>src/middlewares/</code>	Authentification, validation et gestion des erreurs
<code>src/services/</code>	Logique métier réutilisable
<code>src/config/</code>	Variables d'environnement, base de données et Swagger
<code>src/validators/schemas.ts</code>	Schémas de validation Joi
<code>src/utils/</code>	Classes utilitaires telles que <code>`AppError`</code>
<code>prisma/schema.prisma</code>	Schéma de données Prisma

5.2. Dépendances et rôle des bibliothèques

- Express : framework HTTP utilisé pour les routes et les middlewares.
- TypeScript : typage statique, sécurisation du code et meilleure maintenabilité.
- Prisma : abstraction de la base de données, migrations et accès robuste aux données.
- PostgreSQL : stockage relationnel des utilisateurs, recettes, cookbooks et plans.
- Joi : validation des entrées utilisateur et des objets reçus en requête.
- JWT : authentification stateless entre le client et l'API.
- Swagger UI / Swagger JSDoc : documentation interactive de l'API.
- Socket.IO : prise en charge de fonctionnalités temps réel si le projet évolue.

5.3. Gestion des erreurs

La gestion des erreurs est centralisée afin de conserver des réponses cohérentes et exploitables côté client.

- `AppError` pour les erreurs métiers et applicatives.
- `error.middleware.ts` pour le traitement global.
- Conversion des exceptions Prisma en réponses HTTP explicites.
- Erreurs de validation Joi renvoyées avec un statut 400 lisible.

Code HTTP	Signification
400	Données invalides
401	Non authentifié
403	Permission refusée
404	Ressource introuvable
409	Conflit ou duplication
500	Erreur interne

Objectif Standardiser le comportement de l'API et faciliter le débogage.

5.4. Validation des entrées

Les données reçues sont validées avec Joi. Les schémas couvrent notamment l'inscription et la connexion, les recettes, cookbooks, membres, plans de repas, profils utilisateurs et identifiants UUID.

Le middleware `validate` contrôle le body, les params et la query, puis nettoie les données avant la poursuite de la requête.

5.5. Authentification et autorisations

L'authentification repose sur JWT. Après inscription ou connexion, le backend retourne un token utilisé dans l'en-tête Authorization :

```
Authorization: Bearer <token>
```

Les routes protégées passent par le middleware `authenticate`. Les accès à certains cookbooks ou recettes sont ensuite contrôlés par `requireCookbookRole`.

- **OWNER** — propriétaire
- **EDITOR** — éditeur
- **COMMENTER** — commentateur
- **READER** — lecteur

5.6. API REST

Ressource	Endpoints
Authentification	POST /api/auth/register POST /api/auth/login
Utilisateur	GET /api/users/me PATCH /api/users/me

Cookbooks	GET /api/cookbooks
	POST /api/cookbooks
	GET /api/cookbooks/:id
	PATCH /api/cookbooks/:id
	DELETE /api/cookbooks/:id
	POST /api/cookbooks/:id/members
Recettes	GET /api/recipes
	POST /api/recipes
	GET /api/recipes/:id
	PATCH /api/recipes/:id
	DELETE /api/recipes/:id
	POST /api/recipes/:id/favorite
Meal plans	POST /api/recipes/:id/comments
	GET /api/meal-plans
	POST /api/meal-plans
	DELETE /api/meal-plans/:id
Données	GET /api/data/export
	POST /api/data/import
Health	GET /api/health

5.7. Swagger

Swagger fournit une documentation visuelle et testable de l'API.

```
http://localhost:4000/api/docs/
```

- Visualiser les endpoints.
- Tester les requêtes HTTP.
- Disposer d'une documentation technique directement exploitable.
- Présenter plus facilement le projet à des examinateurs ou évaluateurs.

La configuration se trouve dans `backend/src/config/swagger.ts` et contient les informations générales de l'API, sa version, ses routes et les composants de sécurité JWT.

6. Frontend

6.1. Setup et environnement

Le frontend est développé avec React et TypeScript, sur une base Vite.

- `frontend/src/App.tsx` comme point d'entrée principal.
- Vite pour l'environnement de développement.
- Nginx pour servir la version produite dans Docker.
- Variables d'environnement `VITE_API_URL` et `VITE_SOCKET_URL`.

6.2. Pourquoi React + Vite ?

- Technologies largement utilisées dans les projets web modernes.
- Cohérence avec les interfaces utilisateur dynamiques.
- Développement rapide avec une structure claire.
- Environnement de développement très réactif grâce à Vite.
- Architecture modulable grâce aux composants et aux états React.

Dans le cadre du projet, le frontend constitue la couche de consommation de l'API backend, avec des interactions simples et un affichage dynamique des données.

7. Base de données et schéma de données

La base de données est PostgreSQL et son accès est géré par Prisma.

Modèle	Champs principaux
User	id, email, passwordHash, displayName, dietaryPreferences, allergies, favoriteCuisines, defaultServings, createdAt, updatedAt
Cookbook	id, name, description, ownerId, createdAt, updatedAt
CookbookMember	id, cookbookId, userId, role
Recipe	id, title, description, preparationMinutes, cookingMinutes, servings, imageUrl, sourceUrl, ingredients, steps, tags, authorId, cookbookId
Comment	id, content, recipeId, authorId
Favorite	id, userId, recipeId
MealPlan	id, plannedFor, mealType, servings, notes, recipeId, userId, cookbookId

7.1. Relations principales

- Un utilisateur peut posséder plusieurs cookbooks.
- Un utilisateur peut rejoindre plusieurs cookbooks en tant que membre.
- Une recette appartient à un auteur et éventuellement à un cookbook.
- Les commentaires, favoris et plans de repas sont liés à des utilisateurs et à des recettes.
- Les permissions sont gérées par l'enum `CookbookRole` : `OWNER`, `EDITOR`, `COMMENTER`, `READER`.

Ce schéma répond aux besoins du MVP et laisse de la place à de futures évolutions.

8. Guide de démarrage

8.1. Avec Docker Compose

Depuis la racine du projet :

```
docker compose up --build
```

Service	Adresse
Frontend	http://localhost:5173
Backend	http://localhost:4000/api
Swagger	http://localhost:4000/api/docs/
PostgreSQL	localhost:5432

8.2. En développement local

Backend :

```
cd backend
npm install
npx prisma generate
npm run dev
```

Frontend :

```
cd frontend
npm install
npm run dev
```

Les principales variables à configurer dans le fichier `.env` à la racine sont :

- DATABASE_URL
- JWT_SECRET
- JWT_EXPIRES_IN
- BACKEND_PORT
- FRONTEND_PORT
- FRONTEND_URL
- VITE_API_URL
- VITE_SOCKET_URL

9. Sécurité et configuration

- Aucun secret n'est codé en dur dans le code source.
- Les mots de passe sont hachés avec bcrypt.
- L'API vérifie les tokens JWT.
- Les routes sécurisées imposent une authentification.
- La validation Joi protège l'application contre les entrées mal formées.
- CORS est limité à l'URL du frontend configurée.
- L'option `x-powered-by` est désactivée.

Configuration Le fichier `.env`` est essentiel au bon fonctionnement du projet, en particulier dans l'environnement Docker.

10. Points forts et limites du projet

10.1. Points forts

- Architecture backend claire et structurée.
- API REST cohérente et documentée.
- Utilisation de Prisma pour l'accès à la base de données.
- Validation forte des entrées.
- Sécurité basée sur JWT.
- Dockerisation du projet.
- Swagger facilitant la démonstration technique.

10.2. Limites

Le projet est fonctionnel et testable sur les fonctionnalités principales, mais reste incomplet par rapport aux ambitions d'un produit finalisé.

- L'interface frontend reste simple et centrée sur les besoins de démonstration.
- Certaines évolutions visuelles ou UX ne sont pas finalisées.
- Le contexte de rattrapage a imposé des contraintes de temps et une priorité donnée aux éléments essentiels.

11. Note sur le rendu incomplet

Contexte du rendu : Ce projet a été réalisé dans le cadre d'un sujet de rattrapage. Étant actuellement en arrêt, je n'ai malheureusement pas réussi à terminer l'ensemble du projet dans le temps imparti.

J'ai pu terminer la partie backend, qui est fonctionnelle, avec notamment la base de données, l'authentification, la gestion des différentes routes et leur documentation avec Swagger.

J'avais également commencé le frontend, mais je n'ai pas eu suffisamment de temps pour le terminer correctement. La version obtenue nécessitait encore du travail et certaines corrections, notamment au niveau de la sécurité. Je ne souhaitais pas rendre un frontend fait rapidement, avec des failles importantes qui auraient pu pénaliser davantage le projet.

J'ai donc préféré rendre uniquement le backend fonctionnel et documenté, plutôt que d'ajouter un frontend incomplet ou mal sécurisé simplement pour avoir une interface à présenter.

Le projet a cependant déjà été préparé pour accueillir le frontend : la structure, la configuration et les éléments nécessaires à son intégration sont présents. Celui-ci pourra donc être ajouté par la suite sans avoir à reprendre entièrement la partie backend.

Le rendu reste donc incomplet puisqu'il manque l'interface utilisateur, mais la partie technique côté backend est fonctionnelle et peut être testée directement grâce à Swagger.